EMBEDDED HACKING

Reverse Engineering the Raspberry Pi Pico 2

Week 9-BN: Binary Ninja Personal — Read, Hack, and Patch Operator Immediates and a DHT11 Float Scale (Raw `.bin`)

ARM Cortex-M33 • Binary Ninja • OpenOCD

October 17, 2026

For educational and defensive security research only

LEGAL DISCLAIMER: The information, tools, and code provided in this repository and course are strictly for educational, research, and defensive purposes only.

You are explicitly prohibited from using any materials contained herein to access, test, modify, or exploit any device, network, or system that you do not own 100% or for which you do not have explicit, documented, and legally binding authorization to interact with.

By using this repository and course, you acknowledge and agree that:

1. Any illegal, unauthorized, or malicious use of this information is solely your responsibility.
2. The author(s) and contributor(s) of this repository and course shall not be held liable for any damages, legal repercussions, criminal charges, or unauthorized actions resulting from the use, misuse, or abuse of the contents herein.
3. You will comply with all applicable local, state, national, and international laws regarding cybersecurity and computer fraud.

IF YOU DO NOT AGREE WITH THESE TERMS, DO NOT USE THIS REPOSITORY AND COURSE.

What You'll Learn This Week

- Build the lesson project with `Release` and get both an `.elf` and a raw `.bin`
- Dump the **ELF symbol map** with `arm-none-eabi-nm` and use it as ground truth
- Load the raw `.bin` into Binary Ninja at `0x10000000`
- Read the six C operator results as **bare instruction immediates** (`movs r1, #50`, `movs r1, #5`, ...)
- Read the DHT11 single-wire driver, including the inlined `gpio_*` and `time_us_32` helpers
- **Break at the arithmetic** `printf` **call** on live silicon and hack the printed value live
- Optionally **hack the format string live** by pointing `r0` at a RAM replacement
- **Resolve the functions in the Binary Ninja GUI** using the ELF symbol map, including the `dht11.c` symbols
- **Patch** the arithmetic immediate (`50 -> 99`) and the DHT11 scaling constant (`0.1f -> 5.0f`)
- **Rename** the printed label by patching the `"arithmetic_operator:"` string
- **Export** the patched image, convert it to UF2, and flash it

How This Guide Works

The build produces two files for the project:

File	What it is	How we use it
`.elf`	The linked image with a full symbol table	Ground truth for every function address and name
`.bin`	The raw flash image, no headers, no symbols	The image we load into Binary Ninja and reverse

The `.bin` is built **from** the `.elf`, so the ELF tells you exactly what is at every address. We use the ELF symbol map to resolve functions in Binary Ninja, and we reverse-engineer the raw `.bin` the way a real extracted firmware image is reversed.

> **Build `Release`, not `Debug`.** Every address in this guide matches the Week 9 lesson, and the Week 9 lesson is a `Release` build. `Release` optimizes the code the same way the original lesson was built: it **inlines** the `static` helpers `print_operator_results`, `compute_arithmetic_ops`, `compute_operators`, and `print_dht11_reading` straight into `main`, and it inlines the whole `dht11.c` static helper chain (`send_start_signal`, `wait_for_level`, `wait_response`, `read_bit`, `read_40_bits`, `validate_checksum`) into `dht11_read`. Every operator result is folded to a bare `movs r1, #imm` immediate. If you build `Debug`, the SDK function addresses move and the helpers stay separate calls, so nothing lines up. Always build `Release` for this lesson.

The order is **dynamic first, static second**:

1. Break on the live target and prove what the code does.
2. Hack it live in the debugger and watch the output change.
3. Resolve the functions in Binary Ninja using the ELF symbol map.
4. Patch the bytes, export, convert, and flash.

Project	Serial output	Also does	The hacks
`0x001a_operators`	six operator lines, then `Humidity: …%, Temperature: …°C`	DHT11 single-wire sensor on GPIO 4	`50 -> 99` (arithmetic immediate), `0.1f -> 5.0f` (DHT11 scale), and `"arithmetic_operator:" -> "hacked_operator:"`

> **Addresses come from your build.** Every address here is from the `Release` build produced in Step 3. Confirm against your own `.elf` with the command in Step 4.

> **The surprise of this week is how small the operator code is.** The C source computes `x * y`, `x > y`, `x << 1`, and `x += 5` in a helper function, but `Release` folds every one of those into a compile-time constant and stores it as a one-byte immediate inside a `movs r1, #imm` instruction. There is no variable in memory to change — the answer is baked into the instruction stream. The DHT11 reading is different: it is a real measurement scaled by a `float` constant (`0.1f`) sitting in the literal pool, and *that* one is a data word you can patch.

> **A note on the sensor reading.** The humidity/temperature numbers depend on the physical DHT11 attached to GPIO 4. This guide's ground truth is about the *code*, not the sensor: the scale constant, where `dht11_read` lives, and which register holds the reading.

Part 1: Build, Flash, and Get the Symbol Map

Step 1: Install the toolchain

Windows x64

- Install the **Raspberry Pi Pico** extension in VS Code. It installs the ARM GNU toolchain, CMake, Ninja, and the Pico SDK.
- Install **Binary Ninja Personal** and complete its license activation.
- Install **PuTTY** for the serial monitor.

macOS Apple Silicon

```
brew install cmake ninja
```

- Install **Binary Ninja Personal** and complete its license activation.
- Install the **Arm GNU Toolchain**, or let the VS Code Pico extension manage it.

Linux x64

```
sudo apt install cmake ninja-build gcc-arm-none-eabi libnewlib-arm-none-eabi git python3 openocd
minicom
```

- Install **Binary Ninja Personal** and complete its license activation.

Step 2: Verify your tools are the right architecture (do not skip this)

On **macOS Apple Silicon**, the most common failure is an Intel `x86_64` tool on your `PATH`:

```
zsh: bad CPU type in executable: cmake
```

You may have **two Homebrews**: the arm64 one at `/opt/homebrew` and the Intel one at `/usr/local`. If `/usr/local/bin` wins, every `brew` tool is x86_64. Check:

```
file "$(which cmake)"
file "$(which ninja)"
file "$(which arm-none-eabi-gdb)"
file "$(which arm-none-eabi-nm)"
file "$(which openocd)"
file "$(which telnet)"
```

All must report `arm64`. If any is `x86_64`, put the Apple Silicon prefix first for the session and check again:

```
export PATH="/opt/homebrew/bin:$PATH"
hash -r
file "$(which cmake)"
```

To make it permanent, add that `export` to `~/.zshrc`. Do not use Rosetta as a fix; OpenOCD and GDB are exactly the kind of programs where a translation layer produces failures that look like debugger bugs.

`telnet` **is special — and optional.** The GDB MI workflow does not need it; it is only used by the command-port fallback. macOS no longer ships `telnet`, and the Homebrew build is often the Intel one, so `telnet 127.0.0.1 4444` fails with `bad CPU type in executable`. Call the Apple Silicon Homebrew explicitly:

```
/opt/homebrew/bin/brew install telnet
```

If you would rather not install anything, macOS ships an arm64 `nc`, which can connect to the same OpenOCD port:

```
nc 127.0.0.1 4444
```

Windows x64 and **Linux x64** do not have this problem. Skip to Step 3.

Step 3: Build the project with `Release`

Run this inside `0x001a_operators/`:

```
cmake -B build -G Ninja -DPICO_BOARD=pico2 -DPICO_PLATFORM=rp2350 -DCMAKE_BUILD_TYPE=Release
cmake --build build
```

Point Binary Ninja at this repository (once). Every console snippet below reads the repo root from `~/.embedded-hacking-repo`, so Binary Ninja never needs a database open and nothing is hardcoded. From the repo root, run once:

macOS / Linux:

```
pwd > ~/.embedded-hacking-repo
```

Windows (PowerShell):

```
(Get-Location).Path | Set-Content "$env:USERPROFILE\.embedded-hacking-repo"
```

Then build from the Binary Ninja console, so the whole build -> patch -> flash loop stays inside Binary Ninja. The console inherits a minimal `PATH` — on macOS just `/usr/bin:/bin:/usr/sbin:/sbin` — so it does not see Homebrew; add your package manager's `bin` first, then run plain `cmake`.

macOS Apple Silicon:

```
import os, subprocess
root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()
os.environ["PATH"] = "/opt/homebrew/bin:" + os.environ["PATH"]  # the console's PATH omits Homebrew
proj = os.path.join(root, "0x001a_operators")
subprocess.run(["cmake", "-B", "build", "-G", "Ninja", "-DPICO_BOARD=pico2",
                "-DPICO_PLATFORM=rp2350", "-DCMAKE_BUILD_TYPE=Release"], cwd=proj)
subprocess.run(["cmake", "--build", "build"], cwd=proj)
```

Linux x64:

```
import os, subprocess
root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()
proj = os.path.join(root, "0x001a_operators")
subprocess.run(["cmake", "-B", "build", "-G", "Ninja", "-DPICO_BOARD=pico2",
                "-DPICO_PLATFORM=rp2350", "-DCMAKE_BUILD_TYPE=Release"], cwd=proj)
subprocess.run(["cmake", "--build", "build"], cwd=proj)
```

Windows x64:

```
import os, subprocess
root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()
proj = os.path.join(root, "0x001a_operators")
subprocess.run(["cmake", "-B", "build", "-G", "Ninja", "-DPICO_BOARD=pico2",
                "-DPICO_PLATFORM=rp2350", "-DCMAKE_BUILD_TYPE=Release"], cwd=proj)
subprocess.run(["cmake", "--build", "build"], cwd=proj)
```

The build directory now contains the pair we need:

- `0x001a_operators/build/0x001a_operators.elf` and `.bin` — the `.bin` is **17484** bytes (`0x444c`)

If the ARM toolchain is not on your `PATH`, add `-DPICO_TOOLCHAIN_PATH=...`:

OS	Typical toolchain path
Windows x64	`C:/Program Files/Arm GNU Toolchain arm-none-eabi/14.2 rel1/bin`
macOS Apple Silicon	`~/.pico-sdk/toolchain/14_2_Rel1/bin`
Linux x64	`/usr`

> **This guide's toolchain lives at** `~/.pico-sdk/toolchain/14_2_Rel1/bin`. All of `arm-none-eabi-nm`, `arm-none-eabi-objdump`, and `arm-none-eabi-gdb` resolved in this document come from there. If your install is elsewhere, `which arm-none-eabi-nm` tells you where to point.

Step 4: Dump the ELF symbol map

This is the ground truth for the whole lesson. Run `arm-none-eabi-nm` on the ELF and keep the output in a terminal or a text file:

macOS Apple Silicon / Linux x64:

```
arm-none-eabi-nm -n --defined-only build/0x001a_operators.elf | grep -E ' [Tt] '
```

Windows x64:

```
arm-none-eabi-nm -n --defined-only build\0x001a_operators.elf | Select-String ' [Tt] '
```

Each line is `address type name`. The `T` / `t` type is a function. Here are the functions this lesson uses. The signatures come from the ELF's DWARF debug info queried with `arm-none-eabi-gdb -batch -ex "ptype <name>"`, so they are exact.

Our code and the startup chain:

Address	ELF symbol	Signature	Role
`0x1000015c`	`_reset_handler`	`void _reset_handler(void)`	reset entry
`0x10000186`	`platform_entry`	`void platform_entry(void)`	calls `runtime_init`, `main`, `exit`
`0x1000019a`	`data_cpy`	`void data_cpy(void*, void*, void*)`	copies `.data` from flash to SRAM
`0x100001e4`	`_init`	`void _init(void)`	runs `.init_array`
`0x10000210`	`frame_dummy`	`void frame_dummy(void)`	C runtime boilerplate
`0x10000234`	`main`	`int main(void)`	the lesson function (all four static helpers inlined)
`0x100002d4`	`dht11_init`	`void dht11_init(uint8_t)`	the `dht11.c` init function
`0x100002f4`	`dht11_read`	`bool dht11_read(float*, float*)`	the `dht11.c` read function (all six static helpers inlined)

The GPIO and timer functions `main` / `dht11.c` reach:

Address	ELF symbol	Signature	Role
`0x10000440`	`gpio_set_function`	`void gpio_set_function(uint, gpio_function_t)`	SDK GPIO function select (UART pins)
`0x1000047c`	`gpio_set_pulls`	`void gpio_set_pulls(uint, bool, bool)`	SDK pull config (`gpio_pull_up` inlined)
`0x100004a4`	`gpio_init`	`void gpio_init(uint)`	SDK GPIO init (`dht11_init` calls it)
`0x10000f00`	`sleep_us`	`void sleep_us(uint64_t)`	SDK microsecond delay (`dht11_read` calls it)
`0x10000fd8`	`sleep_ms`	`void sleep_ms(uint32_t)`	SDK millisecond delay (`main`, `dht11_read`)
`0x100011bc`	`time_us_64`	`uint64_t time_us_64(void)`	SDK microsecond clock (`printf` path)
`0x100011d0`	`busy_wait_us`	`void busy_wait_us(uint64_t)`	UART timing loop
`0x10001250`	`uart_init`	`uint uart_init(uart_inst_t*, uint)`	SDK UART init
`0x10001424`	`clock_get_hz`	`unsigned long clock_get_hz(clock_handle_t)`	UART clock lookup

The stdio/UART and printf chain `main` reaches:

Address	ELF symbol	Signature	Role
`0x1000313c`	`exit`	`void exit(int)`	C runtime exit
`0x10003144`	`runtime_init`	`void runtime_init(void)`	SDK runtime init
`0x10003170`	`stdio_out_chars_crlf`	`void stdio_out_chars_crlf(stdio_driver_t*, const char*, int)`	CRLF output driver
`0x10003280`	`stdio_put_string`	`int stdio_put_string(const char*, int, bool, bool)`	buffered string output
`0x1000336c`	`stdio_set_driver_enabled`	`void stdio_set_driver_enabled(stdio_driver_t*, bool)`	enable the UART driver
`0x10003394`	`stdio_init_all`	`bool stdio_init_all(void)`	SDK serial init
`0x10003424`	`__wrap_puts`	`int __wrap_puts(const char*)`	the `puts` wrapper (`DHT11 read failed`)
`0x10003460`	`__wrap_vprintf`	`int __wrap_vprintf(const char*, va_list)`	printf core
`0x10003524`	`__wrap_printf`	`int __wrap_printf(const char*, ...)`	the `printf` wrapper (all seven calls)

Address	ELF symbol	Signature	Role
`0x100036e0`	`stdio_uart_init`	`void stdio_uart_init(void)`	SDK UART stdio init
`0x10003820`	`strlen`	`size_t strlen(const char*)`	C runtime string length

SDK helpers the `printf` path reaches:

Address	ELF symbol	Signature	Role
`0x10001a44`	`_out_rev`	`unsigned _out_rev(out_fct_type, char*, size_t, size_t, const char*, size_t, unsigned, unsigned)`	reversed-digit output
`0x10001ae0`	`_ntoa_format`	`unsigned _ntoa_format(out_fct_type, char*, size_t, size_t, char*, size_t, bool, unsigned, unsigned, unsigned, unsigned)`	number formatter
`0x10001cb4`	`_out_char`	`void _out_char(char, void*, size_t, size_t)`	single-char sink
`0x100026fc`	`_vsnprintf`	`int _vsnprintf(out_fct_type, char*, size_t, const char*, va_list)`	the format dispatcher
`0x100030e0`	`vfctprintf`	`int vfctprintf(void (*)(char, void*), void*, const char*, va_list)`	printf format engine

Things in this project that have **no symbol of their own**, because the compiler inlined them:

- `print_operator_results`, `compute_arithmetic_ops`, `compute_operators`, `print_dht11_reading` — the four `static` helpers in `0x001a_operators.c` are inlined into `main`, so their bodies appear directly inside `main`.
- `send_start_signal`, `wait_for_level`, `wait_response`, `read_bit`, `read_40_bits`, `validate_checksum` — the six `static` helpers in `dht11.c` are inlined into `dht11_read`, along with `gpio_set_dir`, `gpio_put`, `gpio_get`, and `time_us_32` from the SDK headers. You see `mcrr` (SIO writes), `ldr [.., #4]` (SIO reads), and `ldr [r0, #40]` (TIMERo read) directly inside `dht11_read`.
- `gpio_pull_up` — `static inline` in the SDK, so `dht11_init` compiles to a direct tail-call to `gpio_set_pulls`.

> `dht_pin` **is a** `b` **symbol, not a function.** `arm-none-eabi-nm -n` lists `20000b7c b dht_pin`. That lowercase `b` is a **local data** symbol: the `dht11.c` file-scope `static uint dht_pin;` is parked in RAM `.bss` at `0x20000b7c`. It holds the GPIO number (`4`) after `dht11_init(4)` runs. There is no function there; do not `Y` it with a prototype.

Step 5: Flash and confirm the output

A `.bin` has no headers, so OpenOCD must be told the base address `0x10000000`. From the repository root:

macOS Apple Silicon / Linux x64:

```
./flash.sh 0x001a_operators/build/0x001a_operators.bin
```

Windows x64 (PowerShell):

```
.\flash.ps1 -Bin 0x001a_operators\build\0x001a_operators.bin
```

Or flash from the Binary Ninja console (the console reads the repo root from the marker file, so it works with no database open):

macOS Apple Silicon / Linux x64:

```
import os, subprocess
root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()  # set once (Step 3)
bin_path = os.path.join(root, "0x001a_operators", "build", "0x001a_operators.bin")
log = os.path.join(os.path.dirname(bin_path), "flash.log")
subprocess.run(["pkill", "-TERM", "-f", "openocd"])  # free the probe first
subprocess.Popen([os.path.join(root, "flash.sh"), bin_path],
                 stdout=open(log, "w"), stderr=subprocess.STDOUT, start_new_session=True)
print("flashing in the background; log:", log)
```

Windows x64:

```
import os, subprocess
root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()  # set once (Step 3)
bin_path = os.path.join(root, "0x001a_operators", "build", "0x001a_operators.bin")
log = os.path.join(os.path.dirname(bin_path), "flash.log")
subprocess.run(["taskkill", "/F", "/IM", "openocd.exe"])  # free the probe first
subprocess.Popen(["powershell", "-ExecutionPolicy", "Bypass", "-File",
                  os.path.join(root, "flash.ps1"), "-Bin", bin_path],
                 stdout=open(log, "w"), stderr=subprocess.STDOUT)
print("flashing in the background; log:", log)
```

Wait for `wrote 17484 bytes ...` and `** Verified OK **`. Open a serial monitor at **115200** baud:

- **Windows x64:** PuTTY -> Connection type **Serial**, the Pico's COM port, speed `115200`.
- **macOS Apple Silicon:** `screen /dev/tty.usbmodem* 115200` (quit with `Ctrl-A` then `K`).
- **Linux x64:** `minicom -D /dev/ttyACM0 -b 115200`.

```
arithmetic_operator: 50
increment_operator: 5
relational_operator: 0
logical_operator: 0
bitwise_operator: 10
assignment_operator: 10
Humidity: 51.0%, Temperature: 23.8°C
...
```

Two of these values are *not* what a naive reading of the source predicts, and the build is correct:

- `increment_operator: 5` — `compute_arithmetic_ops` takes `x` **by value**, so the `*incr = x++;` inside the helper increments the helper's private copy. `compute_operators`' own `x` is still `5` afterwards.
- `bitwise_operator: 10` and `assignment_operator: 10` — because `x` is still `5`, `x << 1` is `10` (not `12`), and `x += 5` is `10` (not `11`). The relational and logical operators compare `5 > 10`, so both are `0`.

The humidity and temperature lines come from the physical DHT11 and vary with your sensor.

Part 2: Load the Raw `.bin` into Binary Ninja

Start from a fresh Binary Ninja state. If you already have a `.bndb` for this lesson, **close it and start over**; a stale database keeps old names and patches.

Step 6: Bring the raw `.bin` into Binary Ninja

A raw `.bin` has no headers, so Binary Ninja cannot know where it belongs or what architecture it is. You must supply both. If you just double-click the `.bin`, Binary Ninja may load it at address `0x0` with a guessed architecture, and every address in this lesson will be wrong.

1. Choose `File -> Open with Options...` (do **not** use plain `File -> Open`).
2. Select `0x001a_operators/build/0x001a_operators.bin`.
3. In the loader options, set:
   - **Architecture:** `thumb2` (the ARMv7-M / ARMv8-M Thumb-2 architecture, which covers the Cortex-M33)
   - **Platform:** `thumb2`
   - **Base Address:** `0x10000000` (the XIP flash base)
4. Click **Open**.

Binary Ninja analyzes the image and opens the linear view.

Verify the load before going further. Press `G`, type `0x10000000`, and read the first two words:

```
0x10000000   0x20082000   initial stack pointer
0x10000004   0x1000015d   reset vector (bit 0 = Thumb)
```

If you instead see data at `0x00000000`, or a vector word without bit 0 set, close the tab and repeat with `Open with Options`. The Cortex-M33 only executes Thumb-2, so `thumb2` is the only correct architecture.

> **Console equivalent:**
>
> ```
> import os
> root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()  # set once (Step 3)
> load(os.path.join(root, "0x001a_operators", "build", "0x001a_operators.bin"),
>      options={"loader.imageBase": 0x10000000, "loader.platform": "thumb2"})
> ```

Step 7: Save it as a Binary Ninja database (`.bndb`)

Binary Ninja never writes back into the `.bin`. Your names, comments, types, and patches live in a separate `.bndb` database. Save one now, before you make any changes:

1. Choose `File -> Save As...`.
2. Save it next to the image as `0x001a_operators.bndb`.
3. From now on, save with `File -> Save` (`Cmd+S` on macOS, `Ctrl+S` on Windows/Linux) whenever you rename or patch.

The two files have different roles:

File	Role
`0x001a_operators.bin`	the raw firmware image; Binary Ninja never modifies it
`0x001a_operators.bndb`	your analysis database: names, types, comments, and patches

When you come back later, **open the** `.bndb`, not the `.bin`; that restores all your work. If a database gets messy, delete the `.bndb` and re-import the `.bin` from Step 6 — the firmware is never at risk. You export the patched image out of this view later, in Step 19.

Step 8: The views you will use

- **Linear view:** the disassembly listing. You navigate, read, and patch here.
- **Graph view:** the control-flow graph of the current function.
- **Decompiler (HLIL):** the pseudo-C decompilation.
- **Hex view:** raw bytes, used for patching.
- **Function list:** the sidebar list of every detected function.

Navigation: `G` go to address, `N` rename, `Y` set type or signature, `;` add a comment. Breakpoints are set from the GUI through the GDB MI adapter — see Step 12.

> **macOS function keys:** the top-row `F` keys are usually mapped to system functions. Every step here uses menu paths that work without them.

> **Optional — load the RP2350 SVD for peripheral names.** If you want Binary Ninja to label peripheral registers (TIMER0, SIO, UART0, …) instead of bare addresses, load the RP2350 SVD. It ships with Week 4, so it lives at `WEEK04/rp2350.svd` inside the repository — remember it is in **Week 4**, not this week. This lesson does not need it; the ELF symbol map already names every function.

Part 3: Dynamic — Break at the `printf` Call and Hack Live

Step 9: Start OpenOCD as a live debug server

Make sure no other OpenOCD is running; a forgotten server holds port `3333`.

macOS / Linux:

```
ps aux | grep -i openocd
```

Windows (PowerShell):

```
Get-Process | Where-Object { $_.ProcessName -like '*openocd*' }
```

Stop any leftover server gracefully:

macOS / Linux:

```
pkill -TERM -f openocd
```

Windows (PowerShell):

```
Get-Process openocd -ErrorAction SilentlyContinue | Stop-Process
```

Start the server **parked at** `main`:

macOS Apple Silicon / Linux x64:

```
BP_ADDR=0x10000234 ./debug-server.sh
```

Windows x64 (PowerShell):

```
$env:BP_ADDR="0x10000234"; .\debug-server.ps1
```

Or start it from the Binary Ninja console, freeing the probe first and launching the server in the background so the console returns immediately:

macOS Apple Silicon / Linux x64:

```
import os, subprocess
root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()  # set once (Step 3)
subprocess.run(["pkill", "-TERM", "-f", "openocd"])  # stop any running server first
log = os.path.join(root, "openocd.log")
p = subprocess.Popen([os.path.join(root, "debug-server.sh")], cwd=root,
                     env=dict(os.environ, BP_ADDR="0x10000234"),
                     stdout=open(log, "w"), stderr=subprocess.STDOUT, start_new_session=True)
print("OpenOCD started (pid", p.pid, "); log:", log)
```

Windows x64:

```
import os, subprocess
root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()  # set once (Step 3)
subprocess.run(["taskkill", "/F", "/IM", "openocd.exe"])  # stop any running server first
log = os.path.join(root, "openocd.log")
p = subprocess.Popen(["powershell", "-ExecutionPolicy", "Bypass", "-File",
                      os.path.join(root, "debug-server.ps1")], cwd=root,
                     env=dict(os.environ, BP_ADDR="0x10000234"),
                     stdout=open(log, "w"), stderr=subprocess.STDOUT)
print("OpenOCD started (pid", p.pid, "); log:", log)
```

`Popen` returns in a few milliseconds; the server keeps running in the background. Check `openocd.log` for `Listening on port 3333`, then connect in Step 10.

Wait for:

```
Info : [rp2350.dap.core0] Examination succeed
Startup breakpoint at 0x10000234 (2-byte hardware execute, one-shot).
Info : starting gdb server for rp2350.dap.core0 on 3333
Info : Listening on port 3333 for gdb connections
```

`BP_ADDR` **parks the core at** `main` **before any client connects.** The script arms a 2-byte hardware breakpoint and then does the startup `reset run`, so the core runs from the vector table and stops at your address with no debugger attached yet. When Binary Ninja connects a moment later, the first thing it reads is already the truth: `Stopped at 0x10000234`. This is the whole reason the lab works cleanly — you never have to drive a reset from outside the GUI.

Use the address you actually want to stop at:

What you want to stop at	Address	Command
`main` (once per reset)	`0x10000234`	`BP_ADDR=0x10000234 ./debug-server.sh`
The **loop** — the arithmetic `printf` call, hit every iteration	`0x10000272`	`BP_ADDR=0x10000272 ./debug-server.sh`

```
BP_ADDR=0x10000234 ./debug-server.sh   # park at main
BP_ADDR=0x10000272 ./debug-server.sh   # park in the loop instead
```

```
$env:BP_ADDR="0x10000234"; .\debug-server.ps1   # park at main
$env:BP_ADDR="0x10000272"; .\debug-server.ps1   # park in the loop
```

This startup stop is single-use. OpenOCD flushes breakpoints when a client connects, so this one is gone once Binary Ninja attaches — fine for `main`, which only runs once per reset. Every breakpoint after that is set from the Binary Ninja GUI (Step 12) and is repeatable. To stop at `main` again, restart the server with `BP_ADDR` and reconnect.

Exactly one core. The line must say `core0` and must **not** mention `core1`. Core1 is never started by this firmware; exposing it makes Binary Ninja read core1's reset-state registers, which are not real addresses, and OpenOCD floods the log with `Failed to read memory at 0xf0000000`. The scripts already use `USE_CORE=0`; do not change it.

Windows driver note: the Debug Probe must use the **WinUSB** driver. If OpenOCD reports `unable to open CMSIS-DAP device`, install it with Zadig (select `Debug Probe (CMSIS-DAP)` -> WinUSB).

Step 10: Connect Binary Ninja to the GDB server

1. Make sure the image is open and analyzed (Part 2) and the server from Step 9 is running (parked at `main`).
2. Choose `Debugger -> Connect to Remote Process`.
3. In the **adapter** dropdown, select **GDB MI**.
4. In the **connect** settings group, set **IP Address** to `127.0.0.1` and **Port** to `3333`.
5. Set **Full GDB Executable Path** to the `arm-none-eabi-gdb` from the **Arm GNU Toolchain 14.2.rel1**. It ships for all three hosts, and the Raspberry Pi Pico VS Code extension installs that same 14.2.rel1 toolchain (including `arm-none-eabi-gdb`) on all of them:

OS	`arm-none-eabi-gdb` path
macOS Apple Silicon	`/Applications/ArmGNUToolchain/14.2.rel1/arm-none-eabi/bin/arm-none-eabi-gdb` (or the Pico extension's `~/.pico-sdk/toolchain/14_2_Rel1/bin/arm-none-eabi-gdb`)
Windows x64	`%USERPROFILE%\.pico-sdk\toolchain\14_2_Rel1\bin\arm-none-eabi-gdb.exe` (Pico extension), or `C:\Program Files (x86)\Arm GNU Toolchain arm-none-eabi\14.2 rel1\bin\arm-none-eabi-gdb.exe`
Linux x64	`~/.pico-sdk/toolchain/14_2_Rel1/bin/arm-none-eabi-gdb` (Pico extension), or the `bin/` directory of the extracted `arm-gnu-toolchain-14.2.rel1-x86_64-arm-none-eabi` tarball

6. Click **Accept**.
7. Restrict the GDB memory map (prevents OpenOCD SWD faults and reconnect loops): In the bottom console panel, click the **Debugger Console** tab (or `View -> Tabs -> Debugger Console`) and enter:

```
source pico-gdb-memory.gdb
```

(If not running from the repository root, enter the absolute path to `pico-gdb-memory.gdb`*.)* Confirm `info mem` displays the Flash (`0x10000000 0x10400000 ro`) and SRAM (`0x20000000 0x20082000 rw`) ranges.

> **Why `source pico-gdb-memory.gdb` is required.** Binary Ninja's register widget automatically previews memory pointed to by registers. When a register contains an unmapped address (such as uninitialized data or constants like `0x69000000`), Binary Ninja probes it. Without a bounded memory map, OpenOCD attempts to read that invalid address over SWD, faults, and reconnects the debug port (`Info : SWD DPIDR 0x4c013477`). Sourcing `pico-gdb-memory.gdb` sets `inaccessible-by-default on` in GDB, causing GDB to intercept and reject invalid reads locally before any packet reaches OpenOCD, eliminating SWD reconnect storms.

> **Use the GDB MI adapter.** It launches a real `arm-none-eabi-gdb --interpreter=mi2` and lets Binary Ninja drive it, so breakpoints and stepping go through real GDB — which sends the correct 2-byte breakpoint length and handles step-over itself. Verified working end to end: connect, GUI breakpoints (**Add Hardware Breakpoint...**, hardware execute), **Step Into** / **Step Over**, and register edits. Stops are reported as `Breakpoint` (not `SingleStep`).
>
> **Do NOT have any breakpoints set in Binary Ninja before you connect.** With the GDB MI adapter, attaching while Binary Ninja already has a breakpoint **hangs the session**. Start the server parked with `BP_ADDR` (Step 9), connect, and only add hardware breakpoints *after* the connection is up. This is a Binary Ninja bug; it is the single most common GDB MI failure.
>
> **The GDB executable path matters.** Use the **14.2.rel1** build on every OS (Windows, macOS, Linux). The 13.3.rel1 build did **not** connect in testing.
>
> **This step is temporary.** Vector35 plans to ship a GDB binary with the GDB MI adapter (Vector35/debugger#929, milestone *Langara*). Once that lands, Binary Ninja provides GDB itself and you will not need to set **Full GDB Executable Path** at all.
>
> **Do not pick Corellium.** Binary Ninja's adapter dropdown also lists **Corellium**, which is for Corellium's virtual devices and expects an API token, not a local OpenOCD server. It is not the adapter for this lab. The dropdown is a combo box, so an accidental arrow-key press can land on it — always read the label back and confirm it says **GDB MI** before clicking **Accept**.

> **The adapter and port are not saved in the `.bndb`.** Every time you relaunch Binary Ninja you must re-select **GDB MI**, re-enter port `3333`, and re-set the GDB path.

> **Watch for an off-screen error dialog.** When a connection fails, Binary Ninja pops a `Binary Ninja critical alert` window that can be positioned mostly outside the main window, which makes it look like nothing happened. If the connect seems to do nothing, check your other display.

The target keeps running. Open the **Registers** tab (bug icon) and confirm you see live values. `pc` inside `0x10003xxx` and `sp` just below `0x20082000` are healthy.

> **If `pc` is `0x00000088`, `0x000000ec`, or `sp` is `0xf0000000`, the session is bad.** Restart the server, then restart Binary Ninja (a server restart while attached leaves Binary Ninja in a stale session), and connect again.

Step 11: Find `main` without relying on its address

`main` can move between programs, so we do not guess it. We follow the one fixed path to it. Press `G` and go to `0x10000000`:

```
0x10000000   0x20082000   initial stack pointer (top of SRAM)
0x10000004   0x1000015d   reset vector
```

Bit 0 of a vector is the Thumb bit, so `0x1000015d` means "start at `0x1000015c`". That is `_reset_handler`. Follow the reset path to `0x10000186`, `platform_entry`:

```
10000186:       ldr     r1, [pc, #80]   @ (100001d8 <data_cpy_table+0x38>)
10000188:       blx     r1
1000018a:       ldr     r1, [pc, #80]   @ (100001dc <data_cpy_table+0x3c>)
1000018c:       blx     r1
1000018e:       ldr     r1, [pc, #80]   @ (100001e0 <data_cpy_table+0x40>)
10000190:       blx     r1
10000192:       bkpt    0x0000
10000194:       b.n     10000192 <platform_entry+0xc>
```

The middle `blx` at `0x1000018c` is the call to `main`. `platform_entry` is byte-identical in every project, so `0x1000018c` catches `main` no matter where the linker placed it. The literal pool at `0x100001dc` holds `main | 1`; clearing bit 0 gives `0x10000234`.

Step 12: Set a hardware breakpoint in the GUI

With the **GDB MI** adapter, Binary Ninja sets breakpoints through real GDB, which sends the correct 2-byte length, so you set them **in the UI**. There is no command port here.

> **Why older drafts used the command port.** Binary Ninja's **GDB RSP** adapter is its own minimal RSP client and sends a **1-byte** breakpoint (`Z0,<addr>,1`); the Cortex-M33 FPB comparators need 2 bytes, so OpenOCD rejected it with `only breakpoints of two bytes length supported`. The old workaround was to arm breakpoints by hand over telnet. **The GDB MI adapter does not have this problem** — it drives real `arm-none-eabi-gdb`, which sends the right length. So everything below is done in the GUI. The command port still exists as a fallback (see the end of this step), but you do not need it.

Where you can stop

You want to stop at	Address	How	Repeatable?
`main`	`0x10000234`	The server starts parked there with `BP_ADDR=0x10000234` (Step 9), so Binary Ninja is already stopped at `main` when it connects.	No — `main` runs once per reset.
The arithmetic `printf` call	`0x10000272`	Set a hardware breakpoint in the GUI, then click **Resume**.	Yes — fires on every iteration.
The second `printf` call (increment)	`0x1000027a`	Same.	Yes.
The `dht11_read` call	`0x100002a2`	Same.	Yes.

Set the loop breakpoint in the GUI

1. Press `G`, type the loop address (`0x10000272`), and press Enter.
2. Set a **hardware execution** breakpoint at that address, either way:
   - `Debugger -> Add Hardware Breakpoint...` — a **hardware execute** (`HE`) breakpoint. **Use this one.**
   - click the line and press `F2` (`Debugger -> Toggle Breakpoint`) — a **software** breakpoint. It will **not** work here: the code is in read-only flash, so GDB cannot install it and the core just keeps running.
3. Click **Resume**. The core is already running the loop, so the breakpoint fires on the next iteration. Binary Ninja stops with the PC at the loop address and reports it as a **Breakpoint** — verified: `Stopped (Breakpoint) at 0x10000272`.

> **No breakpoints before you connect.** With GDB MI, a breakpoint set before the connection hangs the session (Step 10). Start parked with `BP_ADDR`, connect, *then* add breakpoints.

Stepping

With the target halted at the breakpoint, **Step Into** (`F7`) and **Step Over** (`F8`) run through real GDB and move the PC. Verified: `0x10000272 -> 0x10003524 -> 0x10003526 -> ...`.

> **Step Over on the raw `.bin` steps *into* calls.** The raw image has no symbol for `__wrap_printf`, so **Step Over** at the `printf` call behaves like **Step Into**. When the lab needs to execute the call and then stop, it moves the breakpoint to the return site and clicks **Resume** instead (Step 13 shows this).

> **Never use Binary Ninja's Restart button.** On RP2350 it resets and halts inside the boot ROM (`pc=0x88`, `sp=0xf0000000`). To reset cleanly, restart the server with `BP_ADDR` and reconnect.

> **If you ever need the command port.** It is still there — `nc 127.0.0.1 4444`, and `bp <addr> 2 hw` still arms a breakpoint, `rbp <addr>` / `rbp all` still remove them. It is the fallback if you switch back to the **GDB RSP** adapter, whose 1-byte breakpoints the GUI cannot set. With GDB MI you do not need it for this lab.

Step 13: HACK IT LIVE — change the printed `arithmetic_operator`

`main` loads the constant `0x32` (50) into `r1` and calls `printf` on every iteration. We break on that call and change it live:

```
1000026e:       movs    r1, #50 @ 0x32
10000270:       ldr     r0, [pc, #68]   @ (100002b8 <main+0x84>)
10000272:       bl      10003524 <__wrap_printf>
10000276:       movs    r1, #5
10000278:       ldr     r0, [pc, #64]   @ (100002bc <main+0x88>)
1000027a:       bl      10003524 <__wrap_printf>
```

1. Press `G`, go to `0x10000272` (the `bl __wrap_printf` for `arithmetic_operator`).
2. Set a **hardware execute** breakpoint there: `Debugger -> Add Hardware Breakpoint...`. (Do not use `F2` — that is a software breakpoint and will not work on read-only flash.)
3. Click **Resume** in Binary Ninja. The target is already running the loop, so the breakpoint fires on the next iteration. Binary Ninja stops with the program counter at `0x10000272`, `r0 = 0x100038e8`, and `r1 = 0x32`.
4. Open the **Registers** widget (bug icon -> **Registers**).
5. Find `r1`. Its value is `0x32` (50), loaded by the `movs r1, #50` at `0x1000026e`.
6. **Set `r1` to `0x63` (99).** From Binary Ninja's Python console (`Plugins -> Python Console`):

   ```
   dbg.set_reg_value("r1", 0x63)
   ```

   `dbg.set_reg_value(name, value)` writes one register (returns `True` on success). You can also right-click `r1` in the **Registers** widget, press `E` (edit), type `63`, and press Enter. The widget may not repaint the value, but the write reaches the target — you confirm it by the printed output in the next steps.
7. **Move the breakpoint past the call.** You want `printf` to run once and then stop, so move the breakpoint from `0x10000272` to the instruction *after* the call, `0x10000276` (the `movs r1, #5` that begins the increment half of the loop): remove the breakpoint at `0x10000272` and set a hardware breakpoint at `0x10000276`. Two reasons not to just click **Step Over** here: a breakpoint left on the current PC re-traps the step, and Binary Ninja's **Step Over** steps *into* `__wrap_printf` on this raw `.bin` because the image carries no symbol for the call. Moving the breakpoint to the return site is deterministic.
8. Click **Resume** in Binary Ninja. The core executes `bl __wrap_printf` with `r1 = 0x63`, so this iteration prints `arithmetic_operator: 99`, then stops at `0x10000276`.
9. Look at your serial monitor — the `screen` session on the Pico's USB serial port — and at the **Target** tab in Binary Ninja:

   ```
   arithmetic_operator: 99
   ```

You changed a running program's output without touching the binary.

Step 13b: HACK THE STRING LIVE — change `arithmetic_operator:` to `hacked_operator:` (optional)

The text `"arithmetic_operator: %d\r\n"` lives in flash (`.rodata`) at `0x100038e8` , and flash is **read-only at runtime** — a debugger write there does not stick. So instead of overwriting the text in place, redirect the pointer: at the `printf` call, `r0` holds the string address, so point `r0` at a replacement string you place in RAM.

1. Arm the breakpoint at the `printf` call and hit it, exactly as in Step 13 steps 1-3. At the stop, `r0 = 0x100038e8` and `r1 = 0x32` .
2. Put the replacement string into free RAM at `0x20080000` from Binary Ninja's **Python console** (`Plugins -> Python Console`) — no command port needed:

   ```
   dbg.write_memory(0x20080000, b"hacked_operator: %d\r\n\x00")
   ```

   `dbg.write_memory(address, bytes)` is Binary Ninja's debugger memory-write API; it returns `True` on success. That writes `hacked_operator: %d\r\n\0` .
3. Point `r0` at that string:

   ```
   dbg.set_reg_value("r0", 0x20080000)
   ```

   (Or right-click `r0` in the **Registers** widget, press `E` , type `0x20080000` , and press Enter.)
4. If you want the value hack too, set `r1` to `0x63` as in Step 13. Then move the breakpoint past the call in the GUI (remove it at `0x10000272` , set one at `0x10000276`) and click **Resume**. The core runs `printf` with `r0` pointing at your RAM string and `r1 = 0x63` , so this iteration prints:

   ```
   hacked_operator: 99
   ```

   then stops at `0x10000276` .

Like the value hack, this is **one iteration only**: the loop reloads `r0` (and `r1`) from flash on every pass, so the next line is `arithmetic_operator: 50` again. The permanent version is the static patch in Step 18c.

Step 14: Why the hack reverts (and why we patch next)

Press **Resume**. The loop branches back to `0x1000026e` , which reloads `movs r1, #50` , so the next line is `arithmetic_operator: 50` . The live edit changed one iteration only. There is no variable in memory to change; the value is baked into the instruction. To make `arithmetic_operator: 99` permanent we must patch the instruction. That is the static pass.

Press **Pause** to stop the output flood.

Step 15: Kill the debugger and OpenOCD

The live hack is done. Do this **before** the static pass.

1. In the **Debugger** sidebar, click the **X** (**Kill**) (or `Debugger -> Kill`) to disconnect Binary Ninja.
2. **Kill does not stop the OpenOCD process** — `debug-server.sh` started it separately, and it keeps running and holding the probe. Stop it from the Binary Ninja console:

   macOS / Linux:

   ```
   import subprocess
   subprocess.run(["pkill", "-TERM", "-f", "openocd"])  # stop the debug server, free the probe
   ```

   Windows:

   ```
   import subprocess
   subprocess.run(["taskkill", "/F", "/IM", "openocd.exe"])  # stop the debug server, free the
   probe
   ```

3. Confirm nothing is left: `pgrep -fl openocd` (macOS/Linux) prints nothing.

From a terminal it is the same: `pkill -TERM -f openocd`, or `Get-Process openocd | Stop-Process` on Windows.

Part 4: Static — Resolve the Functions in Binary Ninja and Patch

Step 16: Resolve the functions in the Binary Ninja GUI

We now name the functions in Binary Ninja using the ELF symbol map from Step 4. Binary Ninja loaded the raw `.bin` with **no symbols**, so every function shows as `sub_<addr>` — resolving means giving each one its real name and signature.

Three keys do all the work:

Key	Binary Ninja action	Use it for
`G`	Go to address	Jump to a function's address
`Y`	**Change Type**	Set the function's signature. The dialog shows the full prototype, so this sets the name *and* the type in one step.
`N`	Rename	Rename only, when you just want the name and not the type

For each function below: `G` to its address, then `Y` **(Change Type)** and type the prototype from the table.

How to resolve a function in Binary Ninja (`Y`)

`Y` is the **Change Type** key, and it is what actually resolves the function — it turns `void sub_10003394()` into `bool stdio_init_all(void)`. The Change Type dialog shows the full declaration (name and type), so typing the prototype sets both:

1. `G` to the function's address. The cursor lands on the function.
2. Press `Y`. In the Change Type dialog, type the prototype from the table exactly — for example `bool stdio_init_all(void)` — and press Enter.

The decompiler header then shows the real prototype, and calls to the function read cleanly instead of `sub_<addr>()`. `N` is only for renaming without touching the type; `Y` alone sets both the name and the type.

If `Y` seems to do nothing, confirm the cursor is on the function, or right-click it and pick **Change Type...**. Binary Ninja parses what you type and silently keeps the old type if it does not parse, so glance at the header after each `Y`.

Worked example: `main`

1. Press `G`, type `0x10000234`, press Enter. The view jumps there; the cursor lands on `sub_10000234`.
2. Press `Y` (Change Type), type `int main(void)`, press Enter. That sets the name to `main` and the type to `int(void)`.

> **Binary Ninja shows `int32_t` where Ghidra shows `int`.** After you set `int main(void)`, the decompiler header may read `int32_t main(void)`. That is the same type — on this platform `int` is 32 bits and Binary Ninja's parser normalises it to `int32_t`. Do not fight it; it is not an error.

Worked example: `dht11_init`

1. `G` -> `0x100002d4`.
2. `Y` -> `void dht11_init(uint8_t pin)`.

This is our own `dht11.c` code. In this build the SDK's `gpio_init` and `gpio_pull_up` are called directly (`gpio_pull_up` as a tail-call to `gpio_set_pulls`), so the function is small.

Worked example: `dht11_read`

1. `G` -> `0x100002f4`.
2. `Y` -> `bool dht11_read(float* humidity, float* temperature)`.

This is the big one: all six `dht11.c` static helpers are inlined here, along with `gpio_set_dir` / `gpio_put` / `gpio_get` / `time_us_32`, so `dht11_read` contains the entire single-wire protocol and the `0.1f` scale.

Worked example: `gpio_init`

1. `G` -> `0x100004a4`.
2. `Y` -> `void gpio_init(uint gpio)`.

Worked example: `gpio_set_pulls`

1. `G` -> `0x1000047c`.
2. `Y` -> `void gpio_set_pulls(uint gpio, bool up, bool down)`.

This is what `gpio_pull_up(4)` in `dht11_init` compiles to.

Worked example: `__wrap_printf`

1. `G` -> `0x10003524`.
2. `Y` -> `int __wrap_printf(const char *fmt, ...)`. Keep the `...` — `printf` is variadic.

> **`printf` in our source is `__wrap_printf` in the binary.** The SDK links our `printf` calls to its `__wrap_printf` wrapper, which forwards to `__wrap_vprintf`. Rename it `printf` if you prefer the lesson's shorthand, but `__wrap_printf` is what the ELF says.

Worked example: `__wrap_puts`

1. `G` -> `0x10003424`.
2. `Y` -> `int __wrap_puts(const char *s)`.

This is the `else` branch of `print_dht11_reading`, which prints `"DHT11 read failed\r\n"`. (`__wrap_puts` is aliased to `stdio_puts` at the same address.)

The rest of the chain is the same two keystrokes per function (`G`, then `Y`). This is **our code plus the library functions it actually calls** — not the whole SDK. `main` calls `stdio_init_all`, `dht11_init`, `__wrap_printf`, `dht11_read`, `__wrap_puts`, and `sleep_ms`, so we follow that chain down.

The call chain for this project:

```
main
├── stdio_init_all ── stdio_uart_init ── gpio_set_function, uart_init, stdio_set_driver_enabled
│                                        └── uart_init ── clock_get_hz, busy_wait_us
├── dht11_init ── gpio_init, gpio_set_pulls        (gpio_pull_up inlined)
├── __wrap_printf ── __wrap_vprintf ── vfctprintf ── _vsnprintf
│   │                                                └── _ntoa_format / _out_rev / _out_char
│   ├── stdio_out_chars_crlf
│   └── time_us_64
├── dht11_read ── sleep_ms, sleep_us
│   └── send_start_signal / wait_for_level / wait_response / read_bit /
│       read_40_bits / validate_checksum / gpio_set_dir / gpio_put /
│       gpio_get / time_us_32   (all inlined; no calls)
└── __wrap_puts ── stdio_put_string
```

Resolve every function in that chain:

Address	Rename to (`N`)	Signature (`Y`)
`0x1000015c`	`_reset_handler`	`void _reset_handler(void)`
`0x10000186`	`platform_entry`	`void platform_entry(void)`
`0x1000019a`	`data_cpy`	`void data_cpy(void*, void*, void*)`
`0x100001e4`	`_init`	`void _init(void)`
`0x10000210`	`frame_dummy`	`void frame_dummy(void)`
`0x10000234`	`main`	`int main(void)`
`0x100002d4`	`dht11_init`	`void dht11_init(uint8_t)`
`0x100002f4`	`dht11_read`	`bool dht11_read(float*, float*)`
`0x10000440`	`gpio_set_function`	`void gpio_set_function(uint, gpio_function_t)`

Address	Rename to (N)	Signature (Y)
0x1000047c	gpio_set_pulls	void gpio_set_pulls(uint, bool, bool)
0x100004a4	gpio_init	void gpio_init(uint)
0x10000f00	sleep_us	void sleep_us(uint64_t)
0x10000fd8	sleep_ms	void sleep_ms(uint32_t)
0x100011bc	time_us_64	uint64_t time_us_64(void)
0x100011d0	busy_wait_us	void busy_wait_us(uint64_t)
0x10001250	uart_init	uint uart_init(uart_inst_t*, uint)
0x10001424	clock_get_hz	unsigned long clock_get_hz(clock_handle_t)
0x1000313c	exit	void exit(int)
0x10003144	runtime_init	void runtime_init(void)
0x10003170	stdio_out_chars_crlf	void stdio_out_chars_crlf(stdio_driver_t*, const char*, int)
0x10003280	stdio_put_string	int stdio_put_string(const char*, int, bool, bool)
0x1000336c	stdio_set_driver_enabled	void stdio_set_driver_enabled(stdio_driver_t*, bool)
0x10003394	stdio_init_all	bool stdio_init_all(void)
0x10003424	__wrap_puts	int __wrap_puts(const char*)
0x10003460	__wrap_vprintf	int __wrap_vprintf(const char*, va_list)
0x10003524	__wrap_printf	int __wrap_printf(const char*, ...)
0x100036e0	stdio_uart_init	void stdio_uart_init(void)
0x10003820	strlen	size_t strlen(const char*)

Resolve the SDK helpers the `printf` path reaches:

Address	Rename to (N)	Signature (Y)
0x10001a44	_out_rev	unsigned _out_rev(out_fct_type, char*, size_t, size_t, const char*, size_t, unsigned, unsigned)
0x10001ae0	_ntoa_format	unsigned _ntoa_format(out_fct_type, char*, size_t, size_t, char*, size_t, bool, unsigned, unsigned, unsigned, unsigned)
0x10001cb4	_out_char	void _out_char(char, void*, size_t, size_t)
0x100026fc	_vsnprintf	int _vsnprintf(out_fct_type, char*, size_t, const char*, va_list)
0x100030e0	vfctprintf	int vfctprintf(void (*)(char, void*), void*, const char*, va_list)

A `void` return type may not stick — here is the fix. Binary Ninja treats `void` as low-confidence, and its analysis can override it with an inferred type — most often `int32_t` on this 32-bit target. It is most visible on `_reset_handler` (a hand-written assembly entry that never returns normally), but it can happen to **any** function whose return type Binary Ninja thinks it can infer.

Setting the full signature with `Y` reproduces the unwanted `int32_t`, and `fn.return_type = ...` fails too. What works is the **return-value** setter:

```
from binaryninja import ReturnValue, Type
fn = bv.get_function_at(0x1000015c)
if fn is not None:
    fn.return_value = ReturnValue(Type.void())
```

That holds `_reset_handler` at `void` even after reanalysis. If it still will not stick, leave it — it does not affect the rest of the lesson.

`dht_pin` **is data, not a function.** `arm-none-eabi-nm -n` lists `20000b7c b dht_pin`. That lowercase `b` is a **local data** symbol: the linker parks the `dht11.c` file-scope static at RAM address `0x20000b7c`. `dht11_init(4)` stores `4` into it. There is no function there; do not `Y` it with a prototype.

Shortcut — resolves name *and* type for every function. Instead of doing `N` + `Y` by hand, paste this into Binary Ninja's Python console (`Plugins -> Python Console`). It sets each function's name and signature programmatically:

```
from binaryninja import Symbol, SymbolType
# The raw .bin has no headers, so these SDK types don't exist. set_user_type()
# re-parses each signature as C, so an undefined name raises
# "SyntaxError: unknown type name '...'". Define them first.
sdk = bv.parse_types_from_string("""
typedef unsigned int uint;
typedef char* va_list;
typedef unsigned long clock_handle_t;
typedef void (*out_fct_type)(char, void*, size_t, size_t);
struct stdio_driver;
typedef struct stdio_driver stdio_driver_t;
struct uart_inst;
typedef struct uart_inst uart_inst_t;
enum gpio_function {
    GPIO_FUNC_XIP = 0, GPIO_FUNC_SPI = 1, GPIO_FUNC_UART = 2, GPIO_FUNC_I2C = 3,
    GPIO_FUNC_PWM = 4, GPIO_FUNC_SIO = 5, GPIO_FUNC_PIO0 = 6, GPIO_FUNC_PIO1 = 7,
    GPIO_FUNC_GPCK = 8, GPIO_FUNC_USB = 9, GPIO_FUNC_NULL = 0x1f,
};
typedef enum gpio_function gpio_function_t;
""")
for name, ty in sdk.types.items():
    bv.define_user_type(name, ty)

# address: (name, signature)
funcs = {
    0x1000015c: ("_reset_handler",            "void _reset_handler(void)"),
    0x10000186: ("platform_entry",            "void platform_entry(void)"),
    0x1000019a: ("data_cpy",                  "void data_cpy(void*, void*, void*)"),
    0x100001e4: ("_init",                     "void _init(void)"),
    0x10000210: ("frame_dummy",               "void frame_dummy(void)"),
    0x10000234: ("main",                      "int main(void)"),
    0x100002d4: ("dht11_init",                "void dht11_init(uint8_t)"),
    0x100002f4: ("dht11_read",                "bool dht11_read(float*, float*)"),
    0x10000440: ("gpio_set_function",         "void gpio_set_function(uint, gpio_function_t)"),
    0x1000047c: ("gpio_set_pulls",            "void gpio_set_pulls(uint, bool, bool)"),
    0x100004a4: ("gpio_init",                 "void gpio_init(uint)"),
    0x10000f00: ("sleep_us",                  "void sleep_us(uint64_t)"),
    0x10000fd8: ("sleep_ms",                  "void sleep_ms(uint32_t)"),
    0x100011bc: ("time_us_64",                "uint64_t time_us_64(void)"),
    0x100011d0: ("busy_wait_us",              "void busy_wait_us(uint64_t)"),
    0x10001250: ("uart_init",                 "uint uart_init(uart_inst_t*, uint)"),
    0x10001424: ("clock_get_hz",              "unsigned long clock_get_hz(clock_handle_t)"),
    0x1000313c: ("exit",                      "void exit(int)"),
    0x10003144: ("runtime_init",              "void runtime_init(void)"),
    0x10003170: ("stdio_out_chars_crlf",      "void stdio_out_chars_crlf(stdio_driver_t*, const
char*, int)"),
    0x10003280: ("stdio_put_string",          "int stdio_put_string(const char*, int, bool,
bool)"),
    0x1000336c: ("stdio_set_driver_enabled",  "void stdio_set_driver_enabled(stdio_driver_t*,
bool)"),
    0x10003394: ("stdio_init_all",            "bool stdio_init_all(void)"),
    0x10003424: ("__wrap_puts",               "int __wrap_puts(const char*)"),
    0x10003460: ("__wrap_vprintf",            "int __wrap_vprintf(const char*, va_list)"),
    0x10003524: ("__wrap_printf",             "int __wrap_printf(const char*, ...)"),
    0x100036e0: ("stdio_uart_init",           "void stdio_uart_init(void)"),
    0x10003820: ("strlen",                    "size_t strlen(const char*)"),
    0x10001a44: ("_out_rev",                  "unsigned _out_rev(out_fct_type, char*, size_t,
size_t, const char*, size_t, unsigned, unsigned)"),
    0x10001ae0: ("_ntoa_format",              "unsigned _ntoa_format(out_fct_type, char*, size_t,
size_t, char*, size_t, bool, unsigned, unsigned, unsigned, unsigned)"),
    0x10001cb4: ("_out_char",                 "void _out_char(char, void*, size_t, size_t)"),
    0x100026fc: ("_vsnprintf",                "int _vsnprintf(out_fct_type, char*, size_t, const
char*, va_list)"),
    0x100030e0: ("vfctprintf",                "int vfctprintf(void (*)(char, void*), void*, const
char*, va_list)"),
}
for addr, (name, sig) in funcs.items():
    bv.define_user_symbol(Symbol(SymbolType.FunctionSymbol, addr, name))
    f = bv.get_function_at(addr)
```

```
    if f is not None:
        f.set_user_type(sig)
```

SDK type names (`stdio_driver_t` , `uart_inst_t` , `gpio_function_t` , plus `uint` , `va_list` , `clock_handle_t` , and `out_fct_type`) are **not** in the raw `.bin` . `set_user_type` re-parses each signature as C, so an undefined name raises `SyntaxError: unknown type name '...'` and stops the loop — it is not harmless. The `sdk` block above defines them first (an opaque `struct` / `enum` / `typedef` is enough to parse). Standard names (`uint8_t` , `uint32_t` , `uint64_t` , `bool` , `size_t`) are built in. If you add a function that uses another SDK type, add a definition for it to that block too.

Step 17: Read `main` in the decompiler

Open the **Decompiler** view on `main` . Once the functions above are typed, it reads roughly:

```
int32_t main(void)
{
    stdio_init_all();
    dht11_init(4);
    while (true) {
        __wrap_printf("arithmetic_operator: %d\r\n", 0x32);   // 50
        __wrap_printf("increment_operator: %d\r\n", 5);       // 5
        __wrap_printf("relational_operator: %d\r\n", 0);      // false
        __wrap_printf("logical_operator: %d\r\n", 0);         // false
        __wrap_printf("bitwise_operator: %d\r\n", 0xa);       // 10
        __wrap_printf("assignment_operator: %d\r\n", 0xa);    // 10
        if (!dht11_read(&hum, &temp)) {
            __wrap_puts("DHT11 read failed\r\n");
        } else {
            __wrap_printf("Humidity: %.1f%%, Temperature: %.1f°C\r\n", hum, temp);
        }
        sleep_ms(2000);
    }
}
```

The `0x32` , `5` , `0` , `0xa` , and `0xa` are the constants we will patch first. Every one is an **immediate in the instruction stream** — there is no variable in memory to change, which is why the patch edits the instruction operand.

Step 18: Patch 1 — change `arithmetic_operator` from 50 to 99

Go to `0x1000026e` :

```
1000026e: 32 21   movs  r1, #50   @ 0x32
```

`movs r1, #imm8` is a 16-bit Thumb instruction. Its encoding is `0x21XX` (`movs r1, #imm8`), stored little-endian as `XX 21` — so the immediate is the byte at the instruction's **own** address, `0x1000026e` . Change `0x32` (50) to `0x63` (99).

> **This is the opposite byte from the Week 7 `movw` patch.** For a 16-bit `movs` , the immediate is the first byte. For the 32-bit `movw` , the low immediate byte was the *third* byte. Always decode the instruction before you decide which byte to touch.

Project	Address	Before	After	Effect
`0x001a`	`0x1000026e`	`32`	`63`	`movs r1, #50` -> `#99` , prints `arithmetic_operator: 99`

Option A — Hex view:

1. Switch to the **Hex** view (`View -> Hex`).
2. Toggle the lock off so editing is enabled.
3. Go to `0x1000026e` and change the byte `32` to `63`.
4. Return to the linear view, right-click the function -> `Reanalyze`.

Option B — Python console:

```
bv.write(0x1000026e, b"\x63")
print(hex(bv.read(0x1000026e, 1)[0]))  # -> 0x63
```

After reanalysis the instruction reads `movs r1, #99`.

Step 18b: Patch 2 — change the DHT11 scale from `0.1f` to `5.0f`

The DHT11 driver converts the integer and decimal bytes to a float and scales the decimal part:

```
humidity    = data[0] + data[1] * 0.1f
temperature = data[2] + data[3] * 0.1f
```

`dht11_read` computes both with a fused multiply-add against a single `0.1f` constant. Look at the end of `dht11_read`:

```
100003ee:       vmov    s15, r6
100003f2:       vcvt.f32.s32    s12, s15
100003f6:       vmov    s15, r1
100003fa:       vcvt.f32.s32    s14, s15
100003fe:       vmov    s15, r0
10000402:       vcvt.f32.s32    s13, s15
10000406:       vmov    s15, r2
1000040a:       vldr    s11, [pc, #44]  @ 10000438 <dht11_read+0x144>
1000040e:       vcvt.f32.s32    s15, s15
10000412:       movs    r0, #1
10000414:       vfma.f32        s14, s12, s11
10000418:       vfma.f32        s15, s13, s11
1000041c:       vstr    s14, [r5]
10000420:       vstr    s15, [fp]
```

The constant `s11` is loaded by the `vldr s11, [pc, #44]` at `0x1000040a`, which reads the literal-pool word at `0x10000438`. That word is `0x3dcccccd` (float `0.1`), stored little-endian as `cd cc cc 3d`. Change it to `0x40a00000` (float `5.0`), bytes `00 00 a0 40`:

Project	Address	Before	After	Effect
`0x001a`	`0x10000438`	`cd cc cc 3d`	`00 00 a0 40`	`0.1f` -> `5.0f`; the humidity/temperature decimal parts read ~50x larger

```
+-------------------------------------------------------------------+
|  IEEE-754 little-endian change                                    |
|                                                                   |
|  0.1f = 0x3dccccd  ->  bytes  cd cc cc 3d                         |
|  5.0f = 0x40a00000  ->  bytes  00 00 a0 40                        |
|                                                                   |
|  humidity    = int + (decimal * 0.1f)   becomes                   |
|  humidity    = int + (decimal * 5.0f)                             |
|                                                                   |
+-------------------------------------------------------------------+
```

Option A — Hex view:

1. Switch to the **Hex** view (`View -> Hex`).
2. Go to `0x10000438` and change `cd cc cc 3d` to `00 00 a0 40` .
3. Return to the linear view and reanalyze.

Option B — Python console:

```
bv.write(0x10000438, bytes.fromhex("0000a040"))
print(bv.read(0x10000438, 4).hex())  # -> 0000a040
```

> **The scale is applied to both humidity and temperature.** The two `vfma.f32` instructions both read `s11` , so patching the one constant changes both readings, not just temperature.

Step 18c: Patch 3 — rename the printed label

The string `"arithmetic_operator: %d\r\n"` starts at `0x100038e8` . Its first 26 bytes are:

```
61 72 69 74 68 6d 65 74 69 63 5f 6f 70 65 72 61 74 6f 72 3a 20 25 64 0d 0a 00
 a  r  i  t  h  m  e  t  i  c  _  o  p  e  r  a  t  o  r  :     %  d \r \n \0
```

We replace it with `"hacked_operator: %d\r\n\0"` , which is **four bytes shorter**, and pad the tail with NULs:

```
68 61 63 6b 65 64 5f 6f 70 65 72 61 74 6f 72 3a 20 25 64 0d 0a 00 00 00 00 00
 h  a  c  k  e  d  _  o  p  e  r  a  t  o  r  :     %  d \r \n \0 <pad>
```

The `\0` at offset 21 terminates the string, so `printf` stops there and the four padding bytes are never read. The next format string (`"increment_operator: …"`) begins at `0x10003904` , which is 28 bytes past `0x100038e8` , so our 26-byte write cannot reach it.

Project	Address	Before	After	Effect
`0x001a`	`0x100038e8`	`61 72 69 74 68 6d 65 74 69 63 5f 6f 70 65 72 61 74 6f 72 3a 20 25 64 0d 0a 00`	`68 61 63 6b 65 64 5f 6f 70 65 72 61 74 6f 72 3a 20 25 64 0d 0a 00 00 00 00 00`	prints `hacked_operator:` instead of `arithmetic_operator:`

Option A — Hex view:

1. Switch to the **Hex** view (`View -> Hex`).
2. Go to `0x100038e8` and replace the 26 bytes above.
3. Return to the linear view and reanalyze.

Option B — Python console:

```
bv.write(0x100038e8, b"hacked_operator: %d\r\n\x00\x00\x00\x00")
print(bv.read(0x100038e8, 26))  # -> b'hacked_operator: %d\r\n\x00\x00\x00\x00'
```

> **Keep the NUL.** `printf` walks the format string until it hits `\0`. If you omit it, the printer runs on into the padding and then into the next string, producing garbage. The four padding bytes guarantee the terminator.

Step 19: Export the patched `.bin`

```
import os
seg = next(s for s in bv.segments if s.data_length)  # the loadable image segment
data = bv.read(seg.start, seg.data_length)  # base + size come from the view itself
out = os.path.join(os.path.join(root, "0x001a_operators", "build"), "0x001a_operators-h.bin")
open(out, "wb").write(data)
print(len(data), out)  # -> 17484 /.../build/0x001a_operators-h.bin
```

Where the two numbers come from — nothing is hardcoded:

- `seg.start` is the image base Binary Ninja loaded the `.bin` at (`0x10000000`), the same value you pass to `uf2conv --base`.
- `seg.data_length` is the segment's size in the file (`0x444c` = 17484). Exactly one segment carries data (the image); every peripheral and synthetic segment has `data_length == 0`, so `next(...)` picks the image.
- Reading `seg.start` for `seg.data_length` bytes therefore grabs exactly the image.

Two gotchas this avoids:

- **No relative path.** Binary Ninja's Python console runs with a read-only working directory (inside the app bundle), so `open("0x001a_operators-h.bin", "wb")` fails with `OSError: [Errno 30] Read-only file system`. `root` (from `~/.embedded-hacking-repo`, Step 3) is the repo, so the file is written into the project's `build/` — no machine-specific path and no database needed.
- **Read the image, not the whole view.** `bv.read(bv.start, bv.length)` spans the entire mapped range, which is not the image. The segment's `data_length` is the image size.

A different size means you exported a partial view.

Step 20: Convert to UF2

Run from the project directory:

macOS Apple Silicon / Linux x64:

```
python3 ../uf2conv.py 0x001a_operators-h.bin \
  --base 0x10000000 --family 0xe48bff59 --output hacked.uf2
```

Windows x64:

```
python ..\uf2conv.py 0x001a_operators-h.bin ^
  --base 0x10000000 --family 0xe48bff59 --output hacked.uf2
```

> **Or convert from the Binary Ninja console** — it is a normal Python interpreter, so you never have to leave the app. `chdir` to a writable directory first (the default one is read-only), then run the script:
>
> ```
> import os, sys, runpy
> os.chdir(os.path.join(root, "0x001a_operators", "build"))   # the project build dir (writable)
> sys.argv = ["uf2conv.py", "0x001a_operators-h.bin",
>             "--base", "0x10000000", "--family", "0xe48bff59", "--output", "hacked.uf2"]
> runpy.run_path("../../uf2conv.py", run_name="__main__")   # path to your uf2conv.py
> ```
>
> This writes `hacked.uf2` next to the `.bin`, ready to drag onto the Pico.

Step 21: Flash and verify

Hold **BOOTSEL**, plug in the Pico 2, and drag `hacked.uf2` onto the `RP2350` drive. Open the serial monitor:

```
hacked_operator: 99
increment_operator: 5
relational_operator: 0
logical_operator: 0
bitwise_operator: 10
assignment_operator: 10
Humidity: 255.0%, Temperature: 119.0°C
...
```

- The first line now reads `hacked_operator: 99`.
- The humidity/temperature decimals are scaled by **5.0** instead of **0.1**, so the readings are roughly 50x larger (exact numbers depend on your sensor).

All three changes — one instruction byte, one float word, and one string — with no source code.

Faster: flash over the Debug Probe (no BOOTSEL). The repo's `flash.sh` writes the raw `.bin` straight into XIP flash over SWD (`program <bin> 0x10000000 verify reset exit`), so you never touch BOOTSEL or a UF2. Run it from a terminal (`./flash.sh <bin>`), or from the Binary Ninja console **without freezing it** — use `subprocess.Popen`, which returns immediately, and send OpenOCD's output to a log file. (`subprocess.run` blocks the console until the flash finishes; do not use it here.)

```
import os, subprocess
root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()
bin_path = os.path.join(os.path.join(root, "0x001a_operators", "build"), "0x001a_operators-
h.bin")
log = os.path.join(os.path.join(root, "0x001a_operators", "build"), "flash.log")
subprocess.run(["pkill", "-TERM", "-f", "openocd"])  # free the probe first
p = subprocess.Popen([os.path.join(root, "flash.sh"), bin_path],
                     stdout=open(log, "w"), stderr=subprocess.STDOUT, start_new_session=True)
print("flashing in the background; log:", log)
```

The `pkill` frees the probe first; on Windows use `subprocess.run(["taskkill", "/F", "/IM", "openocd.exe"])`.

The console is free the moment this returns. Check it with `print(p.poll())` (`None` = still running, `0` = done) or read `flash.log` — success ends with `** Verified OK **`.

The same non-blocking form without the script:

```
import os, subprocess
ocd = os.path.expanduser("~/.pico-sdk/openocd/0.12.0+dev")
bin_path = os.path.join(os.path.join(root, "0x001a_operators", "build"), "0x001a_operators-
h.bin")
log = os.path.join(os.path.join(root, "0x001a_operators", "build"), "flash.log")
subprocess.run(["pkill", "-TERM", "-f", "openocd"])  # free the probe first
p = subprocess.Popen([f"{ocd}/openocd", "-s", f"{ocd}/scripts",
    "-f", "interface/cmsis-dap.cfg", "-f", "target/rp2350.cfg",
    "-c", "adapter speed 5000",
    "-c", f"program {bin_path} 0x10000000 verify reset exit"],
    stdout=open(log, "w"), stderr=subprocess.STDOUT, start_new_session=True)
print("flashing in the background; log:", log)
```

The Debug Probe is single-owner. If Binary Ninja is still attached (the `debug-server.sh` OpenOCD is running), the flash cannot grab the probe. Detach in Binary Ninja and stop that OpenOCD first:

```
# macOS / Linux
pkill -TERM -f openocd
```

```
# Windows
Get-Process openocd -ErrorAction SilentlyContinue | Stop-Process
```

Success looks like `Programming Finished` -> `Verified OK` -> `Resetting Target`. On Windows use `flash.ps1` (`.\flash.ps1 -Bin <path>`) the same way.

Cheatsheet

Binary Ninja GUI actions

Action	How
Go to address	`G`
Rename function/symbol	`N`
Set type or signature	`Y`
Add comment	`;`
Open Hex view	`View -> Hex`
Enable hex editing	Toggle the lock in the status bar
Reanalyze after a patch	Right-click function -> `Reanalyze`
Edit a register live	`dbg.set_reg_value("r1", 0x63)` in the Python console (or right-click the register, press `E`, type hex, Enter)
Write a RAM string live	`dbg.write_memory(0x20080000, b"hacked_operator: %d\r\n\x00")`
Set a breakpoint	`Debugger -> Add Hardware Breakpoint...` (hardware execute). Do **not** use `F2` — software breakpoints cannot be written to read-only flash.
Move a breakpoint	Remove it and set it at the new address in the GUI (command-port fallback: `rbp <old addr>` then `bp <new addr> 2 hw`)
Confirm what is armed	The **Breakpoints** widget lists it (command-port fallback: `mdw 0xE0002000 8`, each armed breakpoint shows as `<addr | 1>`)
Apply the ELF symbol map	Paste the Python snippet from Step 16 into the Python Console
Load peripheral names (optional)	Load the RP2350 SVD — it lives at `WEEK04/rp2350.svd`

OpenOCD server and reset

The server runs with `gdb_breakpoint_override hard` so that flash-writes are never attempted. Breakpoints in this lab are set in the Binary Ninja GUI through the **GDB MI** adapter (Step 12). The command-port rows below are the fallback if you use the **GDB RSP** adapter instead.

Action	Command
Connect to the OpenOCD prompt (fallback)	`nc 127.0.0.1 4444` (or `telnet 127.0.0.1 4444`)
Reset and run (command port)	`reset run`
Check core state (command port)	`targets`

Action	Command
Set a breakpoint in the GUI	`Debugger -> Add Hardware Breakpoint...` (hardware execute; `F2` software breakpoints do not work on flash)
(fallback) Add a breakpoint without the GUI	`bp <addr> 2 hw`
Remove one breakpoint	`rbp <addr>` — **address only, no length, no** `hw`
Remove every breakpoint	`rbp all`
Start the server parked at `main`	macOS/Linux: `BP_ADDR=0x10000234 ./debug-server.sh` — Windows: `$env:BP_ADDR="0x10000234"; .\debug-server.ps1` (one-shot)
Start the server parked in the loop	macOS/Linux: `BP_ADDR=0x10000272 ./debug-server.sh` — Windows: `$env:BP_ADDR="0x10000272"; .\debug-server.ps1`
Break on the loop in a running target	set a hardware breakpoint in the GUI at the loop address, then **Resume** — repeatable
Make Binary Ninja stepping work	`rp2350.dap.core0 configure -rtos none` (already in the scripts)
Step without re-trapping	move the breakpoint off the current PC first, then **Step Into**/**Step Over**
Reset without desyncing Binary Ninja	**Detach**, `reset run` on the port, reconnect — never `reset run` while attached

Every address and byte we changed

Project	Address	Before	After	Effect
`0x001a`	`0x1000026e`	`32`	`63`	`movs r1, #50` -> `#99`, prints `arithmetic_operator: 99`
`0x001a`	`0x10000438`	`cd cc cc 3d`	`00 00 a0 40`	DHT11 scale `0.1f` -> `5.0f`, readings ~50x larger
`0x001a`	`0x100038e8`	`61 72 69 74 68 6d 65 74 69 63 5f 6f 70 65 72 61 74 6f 72 3a 20 25 64 0d 0a 00`	`68 61 63 6b 65 64 5f 6f 70 65 72 61 74 6f 72 3a 20 25 64 0d 0a 00 00 00 00 00`	prints `hacked_operator:` instead of `arithmetic_operator:`

The operator / DHT11 memory map

Item	Address	Notes
`dht_pin`	`0x20000b7c`	RAM `.bss`, the `dht11.c` static holding the GPIO number (`4`)
Literal-pool word — humidity/temp format	`0x100002b4`	`0x10003988` — `"Humidity: %.1f%%, Temperature: %.1f°C\r\n"`
Literal-pool word — arithmetic format	`0x100002b8`	`0x100038e8` — `"arithmetic_operator: %d\r\n"`
Literal-pool word — increment format	`0x100002bc`	`0x10003904` — `"increment_operator: %d\r\n"`
Literal-pool word — relational format	`0x100002c0`	`0x10003920` — `"relational_operator: %d\r\n"`

Item	Address	Notes
Literal-pool word — logical format	`0x100002c4`	`0x1000393c` — `"logical_operator: %d\r\n"`
Literal-pool word — bitwise format	`0x100002c8`	`0x10003954` — `"bitwise_operator: %d\r\n"`
Literal-pool word — assignment format	`0x100002cc`	`0x1000396c` — `"assignment_operator: %d\r\n"`
Literal-pool word — failure string	`0x100002d0`	`0x100039b4` — `"DHT11 read failed\r\n"`
`dht11_read` literal-pool word	`0x10000434`	`0x400b0000` — TIMER0 base, used by the inlined `time_us_32`
DHT11 scale constant	`0x10000438`	`0x3dcccccd` — `0.1f`
`dht11_read` literal-pool word	`0x1000043c`	`0x20000b7c` — `&dht_pin`

Raw image facts

Item	Value
Build type	`Release`
Load base address	`0x10000000`
Project size	`17484` bytes (`0x444c`)
Fixed `main` anchor	`0x1000018c` (reset handler middle `blx`)
`main`	`0x10000234`
`dht11_init`	`0x100002d4`
`dht11_read`	`0x100002f4`
`printf` call / return, `arithmetic_operator`	`0x10000272` / `0x10000276`
`arithmetic_operator` format string	`0x100038e8`
`increment_operator` format string	`0x10003904`
`relational_operator` format string	`0x10003920`
`logical_operator` format string	`0x1000393c`
`bitwise_operator` format string	`0x10003954`
`assignment_operator` format string	`0x1000396c`
Humidity/Temperature format string	`0x10003988`
`"DHT11 read failed\r\n"` string	`0x100039b4`
DHT11 scale constant	`0x10000438`
DHT11 GPIO pin	`4`
RP2350 UF2 family ID	`0xe48bff59`

Troubleshooting

My operator values are 12 and 11, not 10 and 10

You are reading the original Week 9 text, not this build. In `0x001a_operators.c`, `compute_arithmetic_ops` receives `x` **by value**, so its `*incr = x++;` never touches `compute_operators`' `x`. That `x` stays `5`, so `x << 1` is `10` and `x += 5` is `10`. Confirm it on your own ELF in Step 4: the immediates in `main` are `#50`, `#5`, `#0`, `#0`, `#10`, `#10`.

Binary Ninja hangs or crashes when you connect (macOS 27)

Three different causes have been seen on this setup; check them in this order.

- **A breakpoint set before connecting.** With the **GDB MI** adapter, if the binary view already has a breakpoint, the session hangs. Start parked with `BP_ADDR`, connect, then add breakpoints (see the next entry).
- **The wrong GDB executable.** Point **Full GDB Executable Path** at the **14.2.rel1** toolchain (Step 10). The 13.3.rel1 build did **not** connect in testing.
- **The LLDB adapter.** A crash report with `libdebuggercore.dylib -> std::terminate() -> abort()` and `liblldb` in the stack is the **LLDB** adapter, not GDB MI. Avoid LLDB on this setup.

Use GDB MI, with the 14.2.rel1 path above. If it still fails, fall back to plain `arm-none-eabi-gdb` against the same server — the addresses and register values are identical to the GUI steps.

If Binary Ninja hangs, force-quit it; the connect dialog has no working Cancel. The static steps (resolve, patch, export, flash) never touch the debugger and always work.

The GUI refuses to set a breakpoint (GDB RSP adapter only)

If you are on the **GDB RSP** adapter, the GUI cannot set breakpoints on this target. That adapter is Binary Ninja's own minimal RSP client and sends a **1-byte** breakpoint (`Z0,<addr>,1`); the Cortex-M33 comparators need 2 bytes, so OpenOCD answers `only breakpoints of two bytes length supported`. It affects every address, both `Toggle Breakpoint` and `Add Hardware Breakpoint`, and the dialog's **Size** field is disabled. `gdb_breakpoint_override` makes no difference.

Fix: use the GDB MI adapter (Step 10). It drives real GDB, which sends the correct length, so GUI breakpoints just work. If you must stay on GDB RSP, arm breakpoints from the command port after connecting (`bp <addr> 2 hw`) — but the lab uses GDB MI and does not need that.

GDB MI hangs when you connect (a breakpoint already existed)

With the **GDB MI** adapter, if Binary Ninja already has a breakpoint set when you connect, the session **hangs**. This is a Binary Ninja bug. The working order is:

1. Start the server parked, e.g. `BP_ADDR=0x10000234 ./debug-server.sh` (Windows: `$env:BP_ADDR="0x10000234"; .\debug-server.ps1`).
2. Connect with the **GDB MI** adapter.
3. Only *then* set hardware breakpoints in the UI.

Never have a breakpoint in the binary view before the GDB MI connection. If it hangs, quit Binary Ninja, restart the server with `BP_ADDR`, and connect again before adding any breakpoints.

Step Into / Step Over does nothing (PC never moves)

Two causes have been seen on this target.

1. **A breakpoint on the current PC re-traps the step.** OpenOCD's step-over-breakpoint logic fails with `Duplicate Breakpoint address` and the PC stays put. Fix: move the breakpoint off the current PC (in the GUI), then step.
2. **The `hwthread` RTOS (GDB RSP adapter only).** With the **GDB RSP** adapter, OpenOCD can log `fake step thread 0` and reply without stepping, because the RP2350 config's `-rtos hwthread` makes the current thread id 1 while Binary Ninja sends thread id 0. Fix: `rp2350.dap.core0 configure -rtos none` (the launcher scripts already pass this). **GDB MI does not hit this.**

To tell them apart, turn on OpenOCD logging (`log_output /tmp/ocd.log` , then `debug_level 3` on the command port) and look for `fake step` versus `Duplicate Breakpoint` .

`zsh: bad CPU type in executable: cmake`

An Intel `x86_64` tool is on your `PATH` on Apple Silicon. Run Step 2: `export PATH="/opt/homebrew/bin:$PATH"` , then `hash -r` . Add it to `~/.zshrc` to make it permanent.

My addresses do not match this guide

You probably built `Debug` . This lesson is a `Release` build. Re-run Step 3 with `-DCMAKE_BUILD_TYPE=Release` . A `Debug` build moves the SDK functions and keeps `print_operator_results` , `compute_operators` , and the `dht11.c` helpers as separate calls, so `main` is not at `0x10000234` and the operator results are not bare immediates.

A breakpoint never fires

First, confirm you actually set one, and that it is a **hardware** breakpoint. With the **GDB MI** adapter, `Debugger -> Add Hardware Breakpoint...` (hardware execute) should land in the **Breakpoints** widget. If nothing lands, or the core keeps running, you probably used `F2` (`Toggle Breakpoint`) — that is a software breakpoint and cannot be written to read-only flash, so it never installs. Also check you are on **GDB MI**, not **GDB RSP** (the GDB RSP adapter cannot set breakpoints on this target at all).

Then check the order and the state:

- **Arm it only after Binary Ninja is connected.** OpenOCD flushes every breakpoint when a client attaches, so anything armed earlier is gone. This also applies to `BP_ADDR` on the startup command line.
- **Verify it is armed:** `mdw 0xE0002000 8` . You should see your address with the low bit set (`0x10000272` -> `0x10000273`). All zeros means nothing is armed — re-read this first, because it distinguishes "not armed" from "armed but never reached".
- **Is the core running?** `poll` on the command port should not report a halt. If it is stopped, click **Resume**.
- **Does the address get reached again?** `main` runs once per reset, so use `BP_ADDR` at startup (Step 9) rather than `reset run` while attached. Loop addresses such as `0x10000272` fire on the next pass with no reset — arm them and click **Resume** in Binary Ninja.
- **With GDB MI the stop is reported as** `Breakpoint` and appears in the **Breakpoints** widget, because GDB really did set it.

I edit `r1` (or another register) and it reverts

`main` reloads the value at the top of every loop iteration — `movs r1, #50` at `0x1000026e` runs right before the `printf` at `0x10000272` . So `r1` is only `0x63` for the instant between your edit and the next pass; then it is `0x32` again. The edit sticks only if the core is **genuinely stopped** at the breakpoint and stays stopped.

If it keeps reverting, the core is running, which almost always means the breakpoint is not installed — usually because it is a **software** breakpoint (`F2`) that cannot be written to read-only flash. Use `Debugger -> Add Hardware Breakpoint...` (hardware execute).

> **The Registers widget is a snapshot, not a live view.** Binary Ninja reads the registers at each stop and shows that snapshot; it does not poll the target, and there is no "refresh registers" command. So a value changed outside Binary Ninja will not appear until the next stop.

The string hack does nothing

The live version needs to run **before** the `printf` call, which means breaking at `0x10000272` and redirecting `r0` while stopped. If you let it resume, the loop reloads `r0` from the literal pool on the next pass and the hack is gone.

Also confirm you wrote a NUL-terminated string. `printf` walks bytes until `*r0 == 0` ; without the trailing `\x00` , it keeps reading RAM garbage.

The patched instruction still shows the old value

Right-click the function and choose `Reanalyze` . Binary Ninja caches the disassembly text; a byte edit does not always trigger a re-lift by itself.

The arith patch corrupts the instruction

You patched the wrong byte. `movs r1, #imm8` is 16-bit and its immediate is the **first** byte: the instruction starts at `0x1000026e` , so the byte to change is `0x1000026e` (`0x32 -> 0x63`). Patching `0x1000026f` (the `0x21` opcode) corrupts the instruction and the core will fault.

The DHT11 reading did not change

Confirm you patched the right word. The scale is the literal loaded by `vldr s11, [pc, #44]` at `0x1000040a` ; that literal lives at `0x10000438` and is `cd cc cc 3d` (`0.1f`). Change it to `00 00 a0 40` (`5.0f`). If your sensor is not connected or not answering, `dht11_read` returns `false` and the firmware prints `DHT11 read failed` instead — the scale patch only matters when a reading succeeds.

The `arithmetic_operator` string hack overran

The replacement must be NUL-terminated. `"hacked_operator: %d\r\n"` is 21 characters; write it plus a `\0` (22 bytes) and pad the remaining four bytes with `00` . The next format string begins at `0x10003904` , 28 bytes past `0x100038e8` , so a 26-byte write stays clear of it. A longer write overwrites `"increment_operator: …"` .

The serial capture is garbage on macOS

Reading `/dev/cu.usbmodem*` with a bare `read()` returns garbage. Set **raw termios at 115200** first: clear canonical/echo flags, set `CLOCAL|CREAD` , and `B115200` on input and output. `screen /dev/cu.usbmodem* 115200` does all of this for you; a script must call `tcsetattr` itself. Once set, the capture reads clean `arithmetic_operator: 50` lines.

It worked for a second, then stopped (Binary Ninja's view desyncs)

This is the most common failure, and it has one main cause: **driving the core from the OpenOCD command port while Binary Ninja is connected.**

- If you send `reset run` from the port while attached, the core resets, runs, and halts at your breakpoint — but Binary Ninja never receives the stop event. Its sidebar keeps showing the *previous* location, so **Step**

and **Resume** act on a stale PC and appear to do nothing.
- If the OpenOCD process dies (or you restart it) while attached, Binary Ninja keeps believing it is connected: the sidebar stays, but the menu shows **Pause** enabled and **Resume**/**Step** disabled because Binary Ninja last saw the target *running*.

Recovery: **Detach, then reconnect.** If Detach does nothing (the connection is already dead), restart Binary Ninja — its menu still shows a session that no longer exists.

Prevention:

- Stop at `main` with `BP_ADDR` on a fresh server start, not with `reset run` while attached.
- For loop addresses, set the breakpoint in the GUI and click **Resume**. Let Binary Ninja be the thing that starts the core.
- If you must reset, **Detach first**, `reset run`, then reconnect.
- Never leave a breakpoint on the PC you are about to step or resume from.

The target "blows past" `main` and stops at `0x10003648` instead

`0x10003648` is `stdio_uart_out_flush`, the UART transmit-FIFO drain loop inside `printf`:

```
10003648:       4b02    ldr     r3, [pc, #8]    @ (10003654 <stdio_uart_out_flush+0xc>)
1000364a:       681a    ldr     r2, [r3]
1000364c:       6993    ldr     r3, [r2, #24]
1000364e:       071b    lsls    r3, r3, #28
10003650:       d4fc    bmi.n   1000364c <stdio_uart_out_flush+0x4>
10003652:       4770    bx      lr
```

That is where the core sits while the UART drains, so the core is running `main`'s loop and simply spends nearly all its time there. The breakpoint at `main` did not fire because `main`'s entry runs exactly **once per reset**. If you arm the breakpoint after the reset, or set it while the target is already running and just resume, the core is already past `main` and will never re-execute it. Either arm the breakpoint **before** resetting, or break inside the loop at `0x10000272`, which fires every iteration.

The console floods with `Failed to read memory` or `SWD DPIDR` reconnects

Two causes:

1. **Core 1 is exposed.** The scripts must run with `USE_CORE=0`. Stop the server, confirm only `core0` is reported, restart, then restart Binary Ninja.
2. **Binary Ninja probed an unmapped address for a register preview.** When a register holds an arbitrary integer or uninitialized value (e.g. `0x69000000`), Binary Ninja tries to preview that memory address, triggering an OpenOCD SWD fault and debug-port reconnect (`SWD DPIDR 0x4c013477`). In the **Debugger Console**, run `source pico-gdb-memory.gdb`. This restricts GDB to valid Flash and SRAM, filtering invalid reads locally so OpenOCD's SWD connection stays completely stable.

`Connect to Remote Process` is greyed out and Pause does nothing

Binary Ninja is in a stale session, usually because the debug server restarted while attached. Quit and reopen Binary Ninja (or the `.bndb`) and connect again.

The decompiler still shows the old value after patching

Right-click the function and choose `Reanalyze`.

Fallback: do the dynamic steps with GDB (macOS 27)

If Binary Ninja's debugger crashes on attach on macOS 27 (see Troubleshooting), you can still do the live hack with the ARM GDB from the toolchain, against the same OpenOCD server. The addresses and register values are identical to the GUI steps.

Start the debug server (Step 9), then in a new terminal:

```
arm-none-eabi-gdb
```

At the `(gdb)` prompt:

```
set architecture armv8-m.main
target extended-remote :3333
hbreak *0x10000272
continue
```

Do **not** run `monitor reset run` before `hbreak`. `0x10000272` is inside `main`'s loop, so the breakpoint fires on the next iteration with no reset. If you reset first, the core runs `main` and you will not catch it.

GDB stops at the `printf` call. Confirm the value, change it, and let it run:

```
info registers pc r0 r1  # pc = 0x10000272, r0 = 0x100038e8, r1 = 0x32
set $r1 = 0x63
stepi
continue
```

The serial monitor prints `arithmetic_operator: 99` for the iteration you changed — the same temporary live hack as editing `r1` in the Binary Ninja Registers widget. When you are done, press `Ctrl-C`, then `detach` and `quit`.

If you specifically want to stop at `main` (`0x10000234`), remember its entry runs only once per reset, so the breakpoint must be armed *before* the reset:

```
monitor reset halt
hbreak *0x10000234
continue
```

If you instead set it while the target is running and just `continue`, you will "blow past" `main` and catch the core inside `printf` — in this build at `0x10003648`, the `stdio_uart_out_flush` UART-drain loop.

`hbreak` sets a hardware breakpoint, which is required for read-only flash. It works from plain GDB because GDB sends the 2-byte length the Cortex-M33 comparators need. Binary Ninja's **GDB MI** adapter goes through the same GDB, so its GUI breakpoints work too; the old **GDB RSP** adapter was the one that sent a 1-byte length and could not set breakpoints here.

Glossary

Term	Definition
AAPCS	ARM Architecture Procedure Call Standard — `r0`-`r3` for the first four arguments, `r0` for the return value
Arithmetic operator	C operators for math (`+`, `-`, `*`, `/`, `%`); here `x * y` folds to `#50`
Assignment operator	Compound assignment (`+=`, `-=`, …); here `x += 5` folds to `#10`
Bitwise operator	Operators on individual bits (`<<`, `>>`, `&`, `|`, `^`); here `x << 1` folds to `#10`
`.bss`	Section for uninitialized (or zero-initialized) static/global variables; `dht_pin` lives here
`.data`	Section for initialized static/global variables; copied from flash to SRAM at boot
DHT11	Low-cost single-wire digital humidity and temperature sensor
`.elf`	Linked image with the symbol table; the ground truth for addresses and names
Fused multiply-add	`vfma.f32 Sd, Sn, Sm` computes `Sd = Sd + (Sn * Sm)` in one operation
Immediate value	A constant embedded directly in an instruction, not fetched from memory
Increment operator	`x++` (post) or `++x` (pre); here `x++` returns the old value, `5`
IEEE-754	Standard for floating-point representation; `0.1f` is `0x3dcccccd`
Literal pool	A block of 32-bit constants that Thumb-2 code reaches with PC-relative `ldr`
Logical operator	Operators combining conditions (`&&`, `||`, `!`); here `false`
`movs`	16-bit Thumb move that loads an 8-bit immediate (0-255); immediate is the first byte
`movw`	32-bit Thumb-2 "move wide" that loads a 16-bit immediate; low imm8 is the third byte
PCF8574	An I2C I/O expander; not used this week (that was the LCD, Week 7)
Relational operator	Comparison operators (`<`, `>`, `==`, `!=`); here `5 > 10` is `false`
`.rodata`	Read-only section for constants and string literals; stays in flash
SIO	Single-cycle I/O block; `gpio_set_dir` / `gpio_put` write it with `mcrr`, `gpio_get` reads it
SVD	System View Description file — register names for a peripheral; the RP2350 one is at `WEEK04/rp2350.svd`
Thumb bit	Bit 0 of a Cortex-M function pointer; selects Thumb instruction mode
TIMER0	RP2350 timer block at `0x400b0000`; `time_us_32` reads its `TIMERAWL` register (`+0x28`)
UF2	USB Flashing Format — the file format the Pico 2 bootloader accepts
Vector table	The first words of flash: initial stack pointer and exception vectors

Remember: the ELF tells you what every address is, and the `.bin` is what you actually patch. Prove the behavior dynamically, read `r1` at the arithmetic `printf` call, resolve the names from the ELF (including the `dht11.c` symbols), then patch the bytes — the `movs` immediate (`0x32 -> 0x63`), the DHT11 float word (`0.1f -> 5.0f`), and the format string (`arithmetic_operator:` -> `hacked_operator:`) — and flash.